Typed Decisions in the Browser: What Limits Logit Readout on Sub-1B Language Models

Arnav Dadarya
[AFFILIATION PLACEHOLDER — department, institution]
[PLACEHOLDER — city, country]
arnavdadarya@gmail.com

***Abstract*—Many decisions that software delegates to a language model are small and closed: route a ticket, judge whether a claim is supported. Instead of generating text and parsing it, a *typed logit readout* lists the options as lettered slots and takes a softmax over the answer-letter logits at one position. We implement this client-side (transformers.js on ONNX Runtime, Qwen3-0.6B, 4-bit weights) and measure what limits it: (i) batched branch evaluation, which makes the server-side form efficient, is rejected by ONNX Runtime's fused attention kernel, so criteria are scored serially; (ii) shared-prefix KV reuse is bit-exact in fp32; (iii) on the public 144-row *authored144* fixture, generation and readout score identically (0.419 mean family balanced accuracy): the readout buys closed support, determinism and a per-option distribution, not accuracy; (iv) averaging over option orderings raises the score to 0.483 (cyclic) or 0.442 (full permutation), which an exact McNemar test cannot distinguish at $n = 144$ ($p = 0.23$), while 87 of 144 rows still change their answer under rotation. Debiasing does not close the gap to the 0.813 published for a 4B model. No fine-tuned, trimmed or pruned model was evaluated.**

***Index Terms*—language models, multiple-choice position bias, logit readout, contextual calibration, permutation averaging, KV-cache reuse, on-device inference, WebGPU, negative result**

I. Introduction

A large share of the decisions that applications ask a language model to make are small and closed: *which queue should handle this ticket, is this claim supported by the evidence, does this request need approval*. The conventional implementation asks a chat model to emit JSON and parses it back into control flow. That spends decode steps on tokens the caller discards and, without constrained decoding [1], gives no structural guarantee that the emitted value is one of the permitted ones.

An alternative is to present the options as lettered slots and read the probability distribution over the token ids of those letters at a single position [2]. Nothing is sampled, so nothing can be malformed, and the distribution ranges only over the options supplied. Existing open implementations of this pattern were developed and benchmarked server-side on a GPU [3], [4].

Our question is what the technique is worth when it runs *on the user's device*, where there is no server, no per-token fee, and the text never leaves the tab—and, separately, what limits its accuracy at the model sizes that fit there. This is a short empirical paper. It does not propose a new model or a new debiasing method; permutation averaging and contextual calibration are prior work [5], [6]. Its value is in the measurements, including the ones that came out against us.

Contributions.

1) A browser-native implementation of typed logit readout in which answer-slot token ids are *discovered* from the tokenizer at run time rather than hardcoded, and in which shared-prefix KV reuse is verified bit-exact in fp32 (Sec. III).
2) A negative engineering result: batched branch evaluation against a shared past KV cache is rejected by ONNX Runtime's `GroupQueryAttention` kernel, so the "flat in the number of criteria" scaling of the server-side form does not transfer to this browser stack (Sec. IV).
3) A controlled ablation on a public fixture showing that (a) generation and readout score *identically*, (b) option-order sensitivity is large—87 of 144 decisions change under rotation—and invisible to plain accuracy, and (c) cyclic permutation averaging recovers 6.4 points, which is not statistically separable from full permutation averaging and does not close the gap to larger models (Sec. VII).
4) A cost argument, specific to prefix-reused readout, for why K-ordering debiasing costs K short suffixes rather than K full prompts, with a measured linear suffix-cost model (Sec. V, VII-F).
5) `typedecide`, an open library for bringing one's own decision data, fine-tuning on the answer token only, evaluating with order-robust metrics and exporting to the browser (Sec. VIII). We state up front that *no fine-tuned model has been trained or evaluated with it for this paper*.

II. Background and Related Work

A. Typed logit readout

Given a state s, a question q and options $o_1, \ldots, o_n$, the prompt lists the options as `A.`, `B.`, … and ends at `Answer:`. Let t_i be the token id the tokenizer produces for letter i *at that exact prompt boundary*, and $\ell \in \mathbb{R}^{|V|}$ the logits at the final position. The decision is the restricted softmax

$$P(o_i \mid s, q) \;=\; \frac{\exp \ell_{t_i}}{\sum_{j=1}^{n} \exp \ell_{t_j}}. \tag{1}$$

Two properties follow directly: the support is exactly the supplied options, and the result is deterministic. Robinson

et al. [2] studied this "multiple-choice prompting" against cloze-style scoring of option text; scoring the text directly suffers from surface-form competition [7] and from multi-token options that share prefixes. We therefore score letters, never option text. Constrained (guided) decoding [1] is the generation-side route to the same closed-support guarantee; we did not benchmark it.

B. Prior implementations

The letter-slot readout, the practice of verifying the shared prefix at token level, and the public fixture and metric used in Sec. VI come from prior open-source work [3], which also ships a browser demo built on wllama [8]. It runs BF16 server-side and reports that shared-prefix scoring is not bit-exact in that format [3]. We did not run the closed hosted service that popularised the interface [4] and make no claim about it.

C. Position bias and calibration in multiple choice

That language models answering multiple-choice questions are sensitive to the order and labelling of options is well documented. Zhao *et al.* [6] showed that few-shot predictions carry systematic biases that can be estimated with a content-free input and divided out (*contextual calibration*). Pezeshkpour and Hruschka [9] measured large accuracy swings from reordering options alone. Zheng *et al.* [5] attribute much of the effect to a prior over option *ID tokens*, and use cyclic and full permutation averaging as (costly) debiasing baselines for their cheaper estimator. We adopt cyclic and full permutation averaging and contextual calibration unchanged; what is specific to our setting is their cost structure under prefix reuse (Sec. V) and their behaviour on a sub-1B model.

D. Prefix sharing

Reusing attention key/value state for a shared prompt prefix is standard in LLM serving, e.g. block-level sharing in PagedAttention [10]. The models we use employ grouped-query attention [11], which is relevant only because the fused kernel implementing it is where the negative result of Sec. IV arises.

III. System Design

The system is a static web page: no backend, no build step, and no network traffic after the weights are cached. Inference is transformers.js [12] (v4.3.0) over ONNX Runtime [13], on WebGPU when available and WebAssembly otherwise. Fig. 1 contrasts it with the generation baseline that the same page also runs.

A. Prompt layout

The prompt is split into a *head* shared by every criterion asked about one state and a per-criterion *tail* (Fig. 2). The head holds a fixed instruction and the evidence; the tail holds the question, the lettered options and the literal `Answer:`. The same prompt-building code is imported by the browser engine and by the benchmark harness, and the Python library mirrors it byte for byte with a CI parity check, because a model fine-tuned against a prompt that deployment never sends degrades silently.

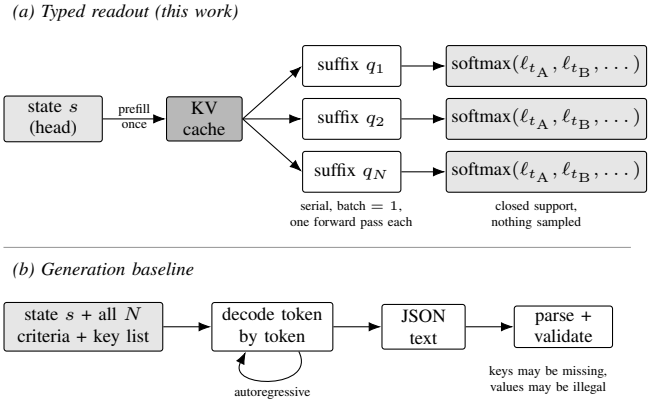


Fig. 1. (a) The state is prefilled once; each criterion is a short suffix scored against the same KV cache and read with a restricted softmax, Eq. (1). Branches are scored serially (Sec. IV). (b) The generation baseline emits one JSON object answering all criteria, which must then be parsed and validated.

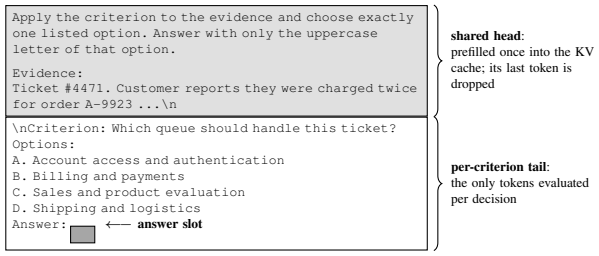


Fig. 2. Prompt anatomy. Grey: shared head. White: per-criterion tail. The readout uses only the logits at the final position.

B. Answer slots are discovered, not assumed

Prior browser implementations ship the answer-letter token ids as configuration as a per-model base id plus the letter index [3]. This is fragile. The ids are not contiguous in any vocabulary we checked: for Gemma 3 [14], the letters `A,B,C,D` resolved to ids $562, 603, 565, 622$, so a `labelBase + index` shortcut (which would give $562, 563, 564, 565$) is silently wrong for every letter after `A`. Whether the natural continuation after `Answer:` is `" A"` or `"A"` also depends on the tokenizer and the preceding character.

For each prompt we therefore probe both separators against the real tokenizer and keep the first for which appending each letter (i) adds exactly one token, (ii) leaves the tokenization of the prompt itself unchanged, and (iii) yields distinct ids; if neither separator satisfies all three, scoring aborts rather than guessing. The cost is a handful of `encode` calls per criterion. The engine also reports the *allowed mass*, the share of the full-vocabulary softmax that falls on the answer letters, as a diagnostic of whether the model has adopted the output format.

C. Shared-prefix KV reuse and the drop-last-token rule

The head is prefilled once and its `present.*` tensors are passed back as `past_key_values` for every criterion, never mutated. Correctness requires the cached ids to be an exact *token-level* prefix of every full prompt, which string concatenation does not guarantee: the first character of the

tail can merge with the last token of the head. Following prior practice [3], we drop the final head token before prefilling, and at scoring time we tokenize the *full* prompt, verify that the cached ids are a prefix of it (aborting otherwise), and feed the remainder as the suffix.

D. Serial branch scoring

Each criterion is one forward pass with batch size one: suffix ids in, one position of logits read. Criteria are independent, so results stream to the caller as they complete; the first decision is available after one prefill and one suffix pass.

IV. A Negative Result: No Batched Branching

The server-side form replicates the prefix KV across a batch and evaluates every criterion in one padded forward pass, which makes total time roughly flat in the number of criteria [3]. We attempted the same through ONNX Runtime and it is refused outright:

```
GroupQueryAttention: batch_size must be
1 when sequence_length > 1 and past
context is given.
```

The fused attention kernel admits a batch wider than one only when *no* past KV is supplied—the one case where batching buys nothing here, because the shared evidence would be re-prefilled for every criterion. wllama, the other common browser runtime, exposes no batched branch evaluation either, for unrelated reasons. With this stack, therefore, criteria are scored serially and total time is linear in the number of criteria. What the shared prefill buys is a *shorter prompt per decision*, not a single forward pass. Claims of "one batched pass, N decisions" belong to a server implementation and do not transfer. We record the error as observed during development; we have not audited other ONNX Runtime versions or non-fused attention exports, either of which might lift the restriction.

V. Debiasing Methods

Let Π be a set of orderings of the n options and $P_\pi(o)$ the probability that Eq. (1) assigns to option *identity* o when the prompt lists the options in order π.

A. Permutation averaging

We average per identity and take the argmax:

$$\bar{P}(o) = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} P_\pi(o), \qquad \hat{o} = \arg\max_o \bar{P}(o). \tag{2}$$

Cyclic averaging uses the n rotations $\pi_k(i) = (i + k) \bmod n$, $k = 0, \ldots, n-1$; *full* averaging uses all $n!$ orderings. Both are balanced in the sense of Fig. 3: each option occupies each slot equally often (once under cyclic, $(n-1)!$ times under full). If the slot prior is additive in probability, $P_\pi(o) = f(o) + g(\mathrm{slot}_\pi(o))$, then

$$\bar{P}(o) = f(o) + \frac{1}{n} \sum_{j=1}^{n} g(j), \tag{3}$$

and the second term is identical for every option, so it cannot change the argmax. We note a caveat: if the prior is instead additive in *logit* space, arithmetic averaging of probabilities cancels it only to first order (the geometric mean would cancel it exactly). Our implementation averages probabilities as in Eq. (2); we did not evaluate log-space averaging.

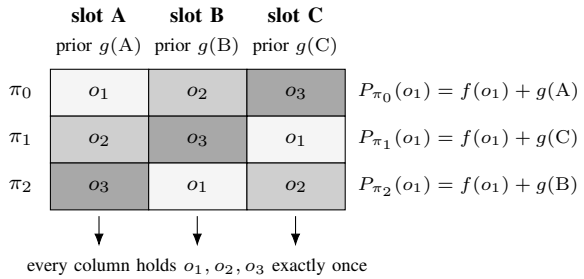


Fig. 3. Cyclic rotation for $n = 3$ is a Latin square over (ordering, slot): each option visits each slot once. Under an additive slot prior g, every option therefore collects the same total $g(\mathrm{A}) + g(\mathrm{B}) + g(\mathrm{C})$ (shown at right for o_1), which shifts all averaged scores equally and cannot change the argmax, Eq. (3).

B. Contextual calibration

Following Zhao *et al.* [6], we estimate the prior of an ordering directly by scoring the same question and ordering with the evidence replaced by a content-free state $s_\varnothing$ (the string `N/A`), and divide it out:

$$\tilde{P}_\pi(o) = \frac{P_\pi(o \mid s) \,/\, \max\big(P_\pi(o \mid s_\varnothing), \epsilon\big)}{\sum_{o'} P_\pi(o' \mid s) \,/\, \max\big(P_\pi(o' \mid s_\varnothing), \epsilon\big)}, \tag{4}$$

with $\epsilon = 10^{-9}$. The null distribution is cached per (question, ordering). *As implemented in our harness, the calibrated mode applies Eq.* (4) *to each of the* n *cyclic orderings and then averages them with Eq.* (2); the row labelled "calibration" in Sec. VII is therefore calibration *combined with* cyclic averaging, not calibration of a single ordering. We did not run single-ordering calibration.

C. Order instability

Whenever more than one ordering is scored, we also record whether the per-ordering argmax *identity* is the same under all of them. A row is *order-unstable* if it is not. This is a property of the model's individual answers; averaging does not change it.

D. Cost: K suffixes, not K prompts

Every ordering changes only the option block, which lives in the tail. With a head of L tokens prefilled once, tails of m tokens, N criteria and K orderings, the readout costs

$$T_{\text{reuse}} = T_{\text{pre}}(L) + NK\, T_{\text{suf}}(m), \tag{5}$$

against $NK\, T_{\text{pre}}(L + m)$ without reuse. A generation-based system has no equivalent: re-asking under K orderings means K complete requests, each with its own decode. Sec. VII-F measures T_{suf}. This is a structural argument supported by that measurement; as stated in Sec. VI, the accuracy harness itself does *not* use prefix reuse, so its wall-clock times are not a demonstration of Eq. (5).

VI. Experimental Setup

Fixture. All accuracy numbers are on the public *authored144* fixture [3]: 144 authored decisions in three families—evidence interpretation, rule application, candidate selection—of 48 rows each, with three options per row, so chance is $1/3$. The fixture varies the position of the gold option across rows (gold is in slot A/B/C for 55/44/45 rows), so a fixed-letter answerer is penalised. We use a public fixture deliberately: it has published numbers for several models, which places our results on a scale we did not build. We do not redistribute it. Its provenance fields mark every row as synthetic, authored, and model-reviewed rather than human-adjudicated.

Metric. Following the fixture's metric contract [3], we report *mean family balanced accuracy*. With families F, gold classes C_f in family f, and $N_{f,c}$ rows of gold class c of which $\mathrm{TP}_{f,c}$ are predicted correctly,

$$\mathrm{MFBA} = \frac{1}{|F|} \sum_{f \in F} \frac{1}{|C_f|} \sum_{c \in C_f} \frac{\mathrm{TP}_{f,c}}{N_{f,c}}. \tag{6}$$

Balanced accuracy [15] is used because classes are uneven (e.g. 21/12/15 in evidence interpretation) and a majority-class answerer should not score well. We also report plain accuracy and the count of order-unstable rows.

Model. We use Qwen3-0.6B [16] in the community ONNX export `onnx-community/Qwen3-0.6B-ONNX` at `q4f16` (4-bit block-quantised weights, fp16 compute), the same artefact the web page loads on WebGPU. It is used as a completion model with no chat template. The accuracy cost of this quantisation relative to fp16/fp32 was not measured.

Prompting. Zero or three demonstrations precede the prompt. The demonstrations are project-authored, not rows of the fixture, and their gold letters are C, B and B (no demonstration has gold A); the first demonstration uses the same three option descriptions as the fixture's evidence-interpretation family. The generation baseline receives the identical prompt, decodes greedily for at most six tokens, and the first capital letter in the output is parsed as the answer.

Harness and hardware. Accuracy runs are made by the script `bench/authored144.mjs` under Node, with transformers.js 4.3.0 on `onnxruntime-node` (CPU), on a single MacBook laptop; the exact model of the machine was not recorded. For simplicity the harness scores every ordering as a *full prompt from scratch*, without prefix reuse; Sec. VII-E establishes that this yields the same probabilities. The cyclic and full-permutation configurations were run twice, with identical accuracy (Sec. VII-C); the others once. Wall-clock times are from single runs on an otherwise uncontrolled laptop. Browser latencies (Sec. VII-G) are from the demo page on WebGPU, also on a MacBook, and are not comparable to the CPU figures.

Comparison numbers. The fixture's authors publish, on the same fixture and metric, 0.440 for Qwen3-0.6B, 0.686 for MiniCPM5-2B and 0.813 for Qwen3.5-4B [3]. These are *their* measurements, on their hardware, scored natively in BF16 with their prompt (their table lists Q8_0 and Q4_K_M GGUF files as the corresponding browser artefacts). We did not reproduce them, and neither numeric format nor prompt is controlled across the comparison.

TABLE I
EIGHT-ITEM ROTATION PROBE (19 SCORED PROMPTS PER ROW; UNIFORM-GUESS ACCURACY ≈0.44). CONSIST.: ITEMS WHOSE ANSWER IS THE SAME UNDER EVERY ROTATION. A SIGNAL, NOT A BENCHMARK.

Model	Style	Acc.	Consist.	Letters chosen
Gemma 3 270M-it	completion	0.421	2/8	A: 89%
Gemma 3 270M-it	chat	0.474	0/8	B: 89%
Gemma 3 1B-it	completion	0.421	2/8	A: 79%
Qwen3 0.6B	completion	0.632	7/8	A 37, B 42, C 21%

TABLE II
QWEN3-0.6B (`q4f16`) ON AUTHORED144; $n = 144$, CHANCE 0.333. K: ORDERINGS PER ROW. UNSTABLE: ROWS WHOSE PER-ORDERING ARGMAX DIFFERS (UNDEFINED FOR $K = 1$). TIME: WHOLE-RUN WALL CLOCK ON AN UNISOLATED LAPTOP CPU, NO PREFIX REUSE; INDICATIVE ONLY (SEE TEXT). ITALIC ROW: PUBLISHED WITH THE FIXTURE [3], NOT MEASURED BY US.

Method	K	MFBA	Plain	Unstable	Time (s)
Readout, 0-shot	1	0.385	0.458	—	56.8
Readout, 3-shot	1	0.419	0.465	—	189.6
Generation, 3-shot	1	0.419	0.465	—	225.3
Qwen3-0.6B, published	*1*	*0.440*	—	—	—
+ full permutation avg.	6	0.442	0.500	106/144	1442.8
+ calibration (with cyclic)	3	0.459	0.514	80/144	918.2
+ cyclic permutation avg.	3	0.483	0.535	87/144	537.3

Significance. For the one ranking that is contested (Sec. VII-C) we use an exact two-sided McNemar test [17], [18] on per-row correctness: with b and c the rows that only one of the two methods gets right, $p = \min\big(1,\, 2\sum_{k=0}^{\min(b,c)} \binom{b+c}{k} 2^{-(b+c)}\big)$.

VII. Results

A. An eight-item probe: position priors at small scale

The measurement that motivated the rest. Eight unambiguous decisions (three with three options, five with two) were scored under every rotation of their option list, 19 prompts in all, so the expected accuracy of a uniform guesser is $(3 \cdot \frac{1}{3} + 5 \cdot \frac{1}{2})/8 \approx 0.44$. Table I shows that both Gemma 3 checkpoints sit at that level while choosing one fixed letter 79–89% of the time, and that going from 270M to 1B parameters did not help; Qwen3-0.6B was order-consistent on 7 of 8 items. Plain accuracy alone would not have revealed the difference—the order-consistency column does. *Eight items is a signal, not a benchmark*; these figures are recorded in the project notes from runs of `bench/rotation_probe.mjs` and no result file for them is committed. The authored144 results below supersede them and, notably, are less flattering to Qwen3-0.6B.

B. Main ablation on authored144

Table II and Fig. 4 give the ablation; Table III breaks it down by family.

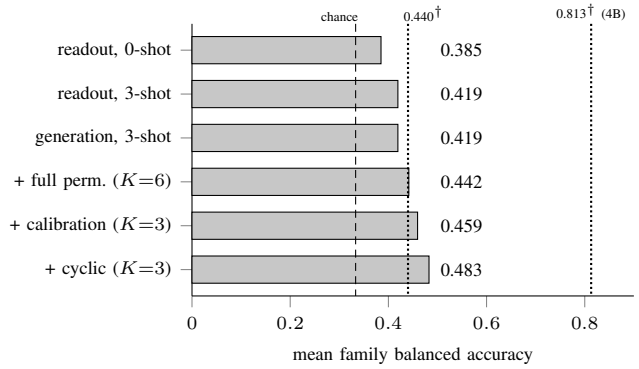


Fig. 4. Ablation on authored144 (bars: measured here). Dashed: chance. Dotted (†): numbers *published* with the fixture [3] for Qwen3-0.6B and for a 4B model, under a different numeric format, prompt and machine; not measured here. The three debiased bars should not be ranked: cyclic and full permutation are not statistically distinguishable (Sec. VII-C).

TABLE III
PER-FAMILY BALANCED ACCURACY FOR THE ROWS OF TABLE II (48 ROWS PER FAMILY).

Method	Candidate sel.	Evidence interp.	Rule appl.
Readout, 0-shot	0.430	0.356	0.369
Readout, 3-shot	0.462	0.435	0.361
Generation, 3-shot	0.462	0.435	0.361
+ full permutation avg.	0.484	0.329	0.512
+ calibration (with cyclic)	0.481	0.400	0.498
+ cyclic permutation avg.	0.523	0.367	0.558

Generation and readout score identically. Both reach 0.41936868… MFBA and 0.465 plain accuracy, with identical per-family values and no unparseable generations (0/144). This is expected: greedy decoding of an answer letter selects the argmax that the readout reads. *The readout therefore does not buy accuracy.* What it buys is closed support (an illegal value cannot be emitted), determinism, and a per-option distribution. We state this plainly because it is the claim a reader is most likely to assume we are making. In this harness the readout run was also $1.19\times$ faster in wall clock ($225.3/189.6$), a modest difference and, given the timing variance noted below, not one we lean on.

The single-ordering readout is close to the published figure. We measure 0.419 at `q4f16`; the published figure is 0.440 in BF16 [3]. The gap is in the direction quantisation would predict, but prompts also differ, so we read this only as a sanity check on the harness, not as a finding.

Cyclic averaging is worth 6.4 points ($0.4828 - 0.4194$), taking the 0.6B model from 0.419 to 0.483. Table III shows the gain is not uniform: rule application rises from 0.361 to 0.558 and candidate selection from 0.462 to 0.523, but evidence interpretation *falls* from 0.435 to 0.367. The per-row predictions of the cyclic run explain why: after the slot prior is averaged away, a *label* prior remains. In evidence interpretation the debiased readout predicts "supported" for 45 of 48 rows (gold: 21), and in candidate selection it predicts the option whose identity is "candidate B" (an option id, not the answer letter) for 40 of 48 (gold: 19). Permutation averaging cannot remove a preference attached to an option's content rather than its slot.

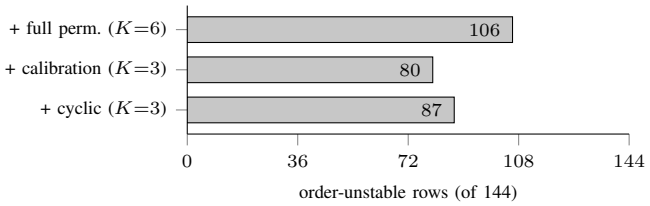


Fig. 5. Order-unstable rows. The $K = 6$ bar is not directly comparable with the $K = 3$ bars (more orderings, more chances to disagree).

The model stays order-sensitive. In the cyclic run, 87 of 144 rows have an argmax identity that differs between rotations. Averaging repairs the *readout*, not the model: of those 87 unstable rows the averaged answer is correct on 39, against 38 of the 57 stable rows (from the per-row prediction file; $39 + 38 = 77$ correct, $77/144 = 0.535$). With calibration applied to each rotation the count is 80 of 144. Under full permutation it is 106 of 144, but that number is not comparable to the other two: six orderings give a row more opportunities to flip than three (Fig. 5). A model that answers partly by position and is corrected afterwards is not the same artefact as one that reads the evidence, and a deployment should not be told otherwise.

C. Cyclic versus full permutation: not distinguishable

Full permutation averaging (six orderings, 0.442) scores *below* cyclic averaging (three orderings, 0.483). The first-order mechanism of Eq. (3) does not predict a difference: both sets place every option in every slot equally often. In plain accuracy the gap is five rows (77 versus 72 of 144). We therefore re-ran both modes saving per-row predictions and applied the exact McNemar test of Sec. VI. Both methods are right on 69 rows and both wrong on 64; of the 11 discordant rows, 8 are correct only under cyclic averaging and 3 only under full permutation, giving a two-sided $p = 0.23$. *This sample cannot distinguish the two procedures. We report both and do not rank them*; in particular we do not claim that cyclic averaging is better than full permutation averaging, or that fewer orderings are preferable. The test concerns plain per-row correctness rather than MFBA, and this is one model and one fixture. The same caution applies to the calibrated variant (0.459), which lies between them and which we did not test.

Two related remarks. First, the re-runs reproduced the earlier aggregates exactly (0.4828 and 0.4416, identical to every printed digit): nothing is sampled, so on a fixed backend the readout is repeatable. Second, the single-ordering runs were made before per-row predictions were being saved, so we have *not* tested whether the 6.4-point gain of cyclic averaging over the single-ordering readout (77 versus 67 rows correct in plain accuracy) is itself statistically significant.

D. Debiasing does not close the capacity gap

The best debiased figure we measure for the 0.6B model is 0.483. The published figures on the same fixture and metric are 0.686 for a 2B model and 0.813 for a 4B model [3] (their numbers, different numeric format, prompt and hardware). Option-order priors are a correctable *share* of small-model error; model capacity remains the dominant variable by a wide margin, and a reader choosing a model should weight it accordingly.

E. Shared-prefix reuse is bit-exact in fp32

`bench/shared_check.mjs` scores five criteria about one state twice—each as a full prompt from scratch, and as suffixes against the once-prefilled head—and reports the maximum absolute difference between the two probability vectors. In fp32 under Node (`onnxruntime-node`, CPU) the maximum drift is 0.000×10^0, i.e. the probabilities are identical, for both Qwen3-0.6B and Gemma 3 270M, with all argmaxes agreeing. This contrasts with the BF16 drift the server-side references report (Sec. II-B) and suggests that the numeric format, not the method, is responsible for that instability. We did not repeat the check on the WebGPU `q4f16` path, where kernels differ, and make no exactness claim there.

F. Cost of a decision

Following the project's own convention, each statement here is labelled MEASURED (observed on hardware), DERIVED (exact arithmetic from the model configuration, with no timing claim) or PROJECTED (an estimate from stated assumptions).

Suffix cost is linear (MEASURED). Holding a 35-token prefilled state fixed and varying only the suffix length (`bench/logit_cost.mjs`; median over repeats, five by default; CPU backend), the six measured points of Fig. 6 fit by least squares to

$$T_{\text{suf}}(m) \approx 28.4 \text{ ms} + 3.23 \text{ ms} \cdot m, \tag{7}$$

with residuals under 2 ms. A typical 45-token criterion is thus PROJECTED at $28.4 + 3.23 \cdot 45 \approx 174$ ms on this backend. For scale only: the accuracy harness, which does not reuse the prefix, averaged $189.6/144 \approx 1.32$ s per three-shot full-prompt pass on the same backend. The two figures come from different prompts and we do not quote their ratio as a speedup.

Most of the returned logits are unread (DERIVED). The stock export has no last-position-only output, so a 45-token suffix returns a tensor of shape $[1, 45, 151936]$: $45 \cdot 151936 \cdot 4 = 27{,}348{,}480$ bytes (27.35 MB, 26.1 MiB) of fp32 per criterion, of which the readout uses n numbers; over one pass of the fixture that is 144×27.35 MB $= 3.94$ GB. The unembedding accounts for 155,582,464 of the model's 596,049,920 parameters, so under the usual estimate of two FLOPs per parameter per token it is 26.1% of per-token compute.

What trimming could buy (bounds, not results). Combining Eq. (7) with the 26.1% share bounds the effect of an export trimmed to the final position: at least $1.27\times$ if only the unembedding FLOPs of the other 44 positions disappear ($174 \rightarrow 137$ ms), and at most $5.5\times$ if the entire per-token term were logits-related ($174 \rightarrow 32$ ms), which it is not, since attention and MLP work are in that term too. The readback of 0.61 MB per position is not bounded by this arithmetic and depends on the backend. *No trimmed export and no head-pruned export has been built or timed*; these are DERIVED bounds. Likewise DERIVED: because Qwen3-0.6B ties its input and output embeddings, pruning the head to the answer letters would remove matmul work and logit traffic but essentially nothing from the download.

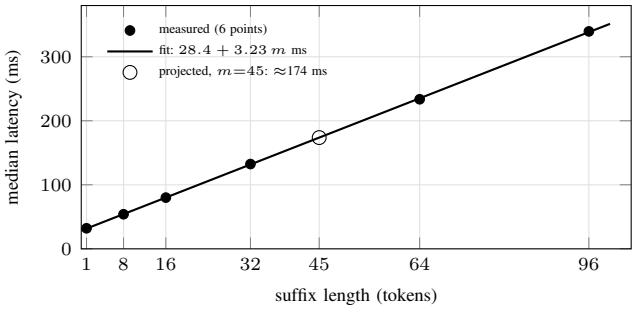


Fig. 6. Suffix latency against suffix length, shared prefix held fixed (Qwen3-0.6B `q4f16`, `onnxruntime-node` CPU). Filled: measured. Open: projection from the fit, not a measurement. Not comparable to WebGPU timings.

G. Browser latency: a small advantage, not a large one

On the demo page (WebGPU, `q4f16`, one reference laptop; informal timings recorded in the project README, not a committed benchmark) a ten-criterion rubric about one ticket took about 1.15 s by readout—roughly 90 ms per decision, with the first decision available at about 310 ms. Given a *carefully engineered* prompt (a worked example plus an explicit list of all required keys), the generation lane answered all ten criteria in about 1.5 s, i.e. the readout was only about $1.3\times$ faster end to end. With a *merely reasonable* prompt that omitted the key list, the same model returned valid JSON that answered only 4 of the 10 criteria and silently dropped the rest. We report both because the honest latency advantage is small; the practical differences are time to first decision, the impossibility of missing or illegal values, and the per-option probabilities.

VIII. The `typedecide` Library

Stock weights know the format but not a deployer's policy. `typedecide` is a Python package for closing that loop; we describe it briefly because it is released with the paper, not because we have results from it. *Bring-your-own-data loaders* read JSONL, JSON, CSV, Parquet and Hugging Face datasets through a column mapping, with strict, diagnostic errors and a validator that flags contradictory labels, class imbalance and inconsistent option sets. *Group splitting*: train/eval splits are made on a grouping key, never on rows, so one state cannot appear on both sides. *Answer-token-only loss*: LoRA [19] training places loss solely on the answer-letter token—the only position the readout reads—truncates long states from the left so the answer slot survives, and randomises option

order so that the model cannot learn "answer A". *Evaluation* implements Eqs. (2)–(6), order consistency, ECE and Brier score. *Export* targets ONNX with optional last-position trimming and head pruning (with the tied-embedding untie-before-slice safeguard). Every training run writes a `manifest.json` recording base model, resolved configuration, a content hash of the dataset, seed, library versions and the resolved answer-slot ids. The package has more than 250 unit tests, which run without a GPU or model download by using stub tokenizers and models; they test the software, not any scientific claim in this paper.

IX. Limitations and Threats to Validity

What was not done. No fine-tuned model has been trained or evaluated; the library's training path is exercised by unit tests only. No trimmed export and no pruned export has been built or timed; every statement about them is DERIVED or PROJECTED. A comparison against hosted frontier-model APIs on the same fixture was scripted (`bench/api_baseline.mjs`) but *not run*; we make no claim about it. Constrained decoding was not benchmarked.

Sample size and repetition. All accuracy results are on 144 rows of one fixture with one model. Differences of a few points between debiasing variants are within noise: the one pair we tested, cyclic versus full permutation, is not distinguishable ($p = 0.23$, Sec. VII-C), and the gain of debiasing over the single-ordering readout was not tested because per-row predictions of those runs were not saved. No confidence intervals are reported. The eight-item probe (Table I) is a signal only, and its outputs are not committed as result files.

Fixture. authored144 is English, synthetic, authored, and model-reviewed rather than human-adjudicated. Nothing here establishes behaviour on real production tickets or other languages. One of our three demonstrations shares option wording with one fixture family.

Uncontrolled quantisation and prompts. Our numbers are `q4f16`; published comparison numbers are BF16 with a different prompt. Differences of a few points across that boundary must not be read as model or method differences.

Latency. Timings are single runs on one unspecified, unisolated laptop, and whole-run wall clock is noisy: the committed full-permutation run averages $1442.8/864 \approx 1.67$ s per scored prompt against $537.3/432 \approx 1.24$ s for the cyclic run, although the two runs score prompts that differ only in option order; and earlier runs of these two configurations, whose result files have since been overwritten, differed from the committed times by roughly 5% and 30%. The Time column of Table II is indicative only. CPU (Node) and WebGPU (browser) figures are not comparable with each other or across machines. Browser timings are informal. The accuracy harness does not use prefix reuse, so Eq. (5) is supported by the suffix-cost measurement, not by an end-to-end timing of debiased readout with reuse.

Probabilities are not calibrated confidence. The readout distribution is conditional on the supplied options. We computed no ECE or reliability analysis on this fixture, and the probabilities should not be thresholded as operational confidence without one.

The negative result is stack-specific. It concerns ONNX Runtime's fused `GroupQueryAttention` as reached through transformers.js 4.3.0; no standalone reproduction script is committed, and other runtimes or exports may behave differently.

Debiasing is partial. Permutation averaging removes slot priors only; the label prior documented in Sec. VII remains, as does the capacity gap.

X. Conclusion and Future Work

Typed logit readout runs in a browser tab on a 0.6B model, at roughly 90 ms per decision on one laptop GPU, with discovered answer slots and bit-exact (fp32) shared-prefix reuse, but without the batched branching that makes the server-side form scale flat. On a public fixture it is exactly as accurate as asking the same model to generate the letter; its advantages are structural. Option-order sensitivity is severe at this scale, invisible to plain accuracy, and partly correctable: cyclic averaging recovers 6.4 points at the price of K short suffixes (full permutation averaging recovers less, but the two are not statistically distinguishable at $n = 144$), yet most rows remain order-unstable, a label prior survives, and the result (0.483) remains far from the 0.813 published for a 4B model. The obvious next steps are the ones we have not taken: fine-tune on order-randomised data with answer-token-only loss and measure whether order instability itself falls; build and time trimmed and pruned exports to convert the DERIVED bounds into measurements; evaluate log-space averaging and single-ordering calibration; run the hosted-API comparison; and repeat on larger and non-synthetic fixtures with confidence intervals.

Reproducibility

Code, benchmark scripts and the result files cited here are in the project repository, https://github.com/ardada2468/typedecide. The script `fetch_fixtures.sh` in `bench/` fetches the fixture from its upstream repository [3], `run_grid.sh` regenerates every row of Table II, and `compare.mjs` runs the McNemar test. Figure sources carry their data inline with the originating result file named in a comment.

Acknowledgment

This work builds on a prior open-source implementation [3], which is not affiliated with and has not endorsed it. Portions of the software, analysis and manuscript text were produced with the assistance of an AI coding assistant (Anthropic's Claude); the author reviewed and takes responsibility for all content.

References

[1] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv preprint arXiv:2307.09702*, 2023.

[2] J. Robinson, C. M. Rytting, and D. Wingate, "Leveraging large language models for multiple choice question answering," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023.

[3] TheoLeeCJ, "openjev," GitHub repository, MIT licence, 2026, [Online]. Available: https://github.com/TheoLeeCJ/openjev. Accessed: Sep. 20, 2026.

[4] archerhume, "Jev's architecture unmasked," Blog post, [Online]. Available: https://archerhume.com/posts/jevs-architecture-unmasked/. Accessed: Sep. 20, 2026.

[5] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "Large language models are not robust multiple choice selectors," in *Proc. 12th Int. Conf. Learning Representations (ICLR)*, 2024.

[6] Z. Zhao, E. Wallace, S. Feng, D. Klein, and S. Singh, "Calibrate before use: Improving few-shot performance of language models," in *Proc. 38th Int. Conf. Machine Learning (ICML)*, ser. Proc. Machine Learning Research, vol. 139, 2021, pp. 12 697–12 706.

[7] A. Holtzman, P. West, V. Shwartz, Y. Choi, and L. Zettlemoyer, "Surface form competition: Why the highest probability answer isn't always right," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[8] ngxson, "wllama," GitHub repository, [Online]. Available: https://github.com/ngxson/wllama. Accessed: Sep. 20, 2026.

[9] P. Pezeshkpour and E. Hruschka, "Large language models sensitivity to the order of options in multiple-choice questions," in *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024.

[10] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proc. 29th ACM Symp. Operating Systems Principles (SOSP)*, 2023.

[11] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[12] Hugging Face, "Transformers.js," Software, version 4.3.0, [Online]. Available: https://github.com/huggingface/transformers.js. Accessed: Sep. 20, 2026.

[13] ONNX Runtime developers, "ONNX Runtime," Software, [Online]. Available: https://onnxruntime.ai. Accessed: Sep. 20, 2026.

[14] Gemma Team, "Gemma 3 technical report," *arXiv preprint arXiv:2503.19786*, 2025.

[15] K. H. Brodersen, C. S. Ong, K. E. Stephan, and J. M. Buhmann, "The balanced accuracy and its posterior distribution," in *Proc. 20th Int. Conf. Pattern Recognition (ICPR)*, 2010, pp. 3121–3124.

[16] Qwen Team, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[17] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[18] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.

[19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. 10th Int. Conf. Learning Representations (ICLR)*, 2022.